

山东大学



网络安全创新创业实践

——基于 OpenFHE BFV-RNS 的密文卷积与旋转次数优化

7.24 作业 5+ 作业 6

姓名	学号	班级
钟晴	202322460127	密码 2 班
宋坤鹏	202322460130	密码 1 班
赵思鹏	202300460075	密码 1 班
田学琛	202300460064	密码 1 班

山东大学网络空间安全学院

目录

1 实验目的	1
2 实验环境	1
3 实验原理及基础知识	1
3.1 全同态加密	1
3.2 BFV 同态加密方案	3
3.3 开源全同态加密库	5
3.4 “打包 → 旋转 → 累加” 策略	6
4 明文槽位模拟	9
4.1 实验输入	9
4.2 槽位布局与模拟实现	10
5 作业 5：基线密文卷积	12
6 作业 6：旋转次数优化与理论分析	16
7 实验结果与分析	19
7.1 正确性验证	19
7.2 操作次数与运行时间	21
8 结论	22

1 实验目的

作业 5: 选择任一开源全同态加密库 (CPU 或 GPU 均可), 使用单输入单输出 4×4 输入和 3×3 卷积核 (步长 1, 无填充), 实现密文卷积并验证结果正确性

作业 6: 采用“打包 $\rightarrow$ 旋转 $\rightarrow$ 累加”策略, 探索本次卷积实现中的旋转次数是否达到理论最小值

本实验选择开源全同态加密库 OpenFHE, 采用 BFV-RNS 整数同态加密方案, 实现单输入单输出的 4×4 输入与 3×3 卷积核的密文卷积。卷积步长为 1, 不进行填充, 因此输出尺寸为 2×2 。在完成正确性验证后, 进一步采用“打包、旋转、明文乘法、累加”策略, 分析并优化密文卷积中的旋转次数。

2 实验环境

本实验在 CPU 环境下完成, 使用的软硬件配置如表 1 所示。实验程序使用 C++ 实现, 通过 CMake 完成编译, 并调用 OpenFHE 提供的 BFV-RNS 接口完成密钥生成、批处理编码、加密、密文旋转、密文-明文乘法、密文加法和解密。

表 1: 实验软硬件环境

项目	配置
操作系统	Ubuntu 虚拟机
执行平台	CPU
编程语言	C++17、Python 3
全同态加密库	OpenFHE 1.5.1
同态加密方案	BFV-RNS
构建工具	CMake
编译模式	Release
安全级别	HEStd_128_classic
明文模数	65537
多项式环维度	8192

3 实验原理及基础知识

3.1 全同态加密

基本概念 传统加密技术主要用于保护数据在存储和传输过程中的机密性。当服务器需要对加密数据进行处理时, 通常需要先对密文解密, 这会导致数据在计算过程中暴露。全同态加密 (Fully Homomorphic Encryption, FHE) 允许计算方直接在密文上执行运算, 而不需要获得明文或解密密钥, 计算所得密文解密后与对应的明文计算结果一致。

设明文为 m_1 和 m_2 , 加密算法为 Enc, 解密算法为 Dec。若加密方案支持同态加法, 则有

$$\text{Dec}(\text{Enc}(m_1) \oplus \text{Enc}(m_2)) = m_1 + m_2,$$

若加密方案支持同态乘法，则有

$$\text{Dec}(\text{Enc}(m_1) \otimes \text{Enc}(m_2)) = m_1 m_2.$$

其中， $\oplus$ 和 $\otimes$ 分别表示密文域中的同态加法和同态乘法。由于加法和乘法能够组合表示一般的算术电路，因此同时支持任意次数加法和乘法的加密方案可以在理论上计算任意可计算函数。

按照支持的运算能力，同态加密通常可以分为以下三类：

- (1) 部分同态加密，只支持一种类型的同态运算，例如只能进行加法或只能进行乘法；
- (2) 有限级同态加密，支持加法和乘法，但只能计算乘法深度受限的电路；
- (3) 全同态加密，可以通过噪声刷新等技术支持任意深度的同态计算。

全同态加密方案 一个典型的全同态加密方案通常包含以下四个基本算法：

- (1) KeyGen：生成公钥、私钥和同态计算所需的评估密钥；
- (2) Enc：使用公钥将明文加密为密文；
- (3) Eval：在密文上执行指定的加法、乘法、旋转等计算；
- (4) Dec：使用私钥解密计算后的密文并恢复明文结果。

设明文输入为 $m_1, m_2, \dots, m_n$ ，待计算函数为 f ，则全同态计算过程可以表示为

$$(pk, sk, evk) \leftarrow \text{KeyGen}(1^\lambda),$$

$$c_i \leftarrow \text{Enc}(pk, m_i), \quad i = 1, 2, \dots, n,$$

$$c_{\text{out}} \leftarrow \text{Eval}(evk, f, c_1, c_2, \dots, c_n),$$

$$m_{\text{out}} \leftarrow \text{Dec}(sk, c_{\text{out}}).$$

其中， λ 表示安全参数， pk 表示公钥， sk 表示私钥， evk 表示评估密钥。全同态加密的正确性要求满足

$$\text{Dec}(sk, \text{Eval}(evk, f, \text{Enc}(pk, m_1), \dots, \text{Enc}(pk, m_n))) = f(m_1, \dots, m_n).$$

在实际应用中，数据所有者负责生成密钥并加密输入，计算服务器只获得密文和评估密钥。服务器在无法获知输入明文的情况下完成计算，最后由持有私钥的数据所有者解密结果。

密文噪声与自举技术 多数全同态加密方案的密文中都包含一定的噪声。噪声是保证方案安全性的重要组成部分，但在同态计算过程中会不断增长。通常，同态加法造成的噪声增长较小，而同态乘法会使噪声显著增加。

设密文 c 所包含的噪声为 $\eta(c)$ ，则可以将不同运算后的噪声增长关系概括为

$$\eta(c_1 \oplus c_2) \approx \eta(c_1) + \eta(c_2),$$

$$\eta(c_1 \otimes c_2) \gg \eta(c_1) + \eta(c_2).$$

当密文噪声超过方案允许的范围时，密文将无法被正确解密。因此，同态计算需要根据计算电路中的连续乘法层数设置参数，该层数称为乘法深度。

为了支持任意深度的密文计算，全同态加密可以采用自举 (Bootstrapping) 技术刷新密文。自举的基本思想是，在密文状态下同态执行解密电路，生成一个噪声较小但对应相同明文的新密文。其过程可以表示为

$$c' = \text{Bootstrap}(c),$$

并满足

$$\text{Dec}(sk, c') = \text{Dec}(sk, c),$$

同时 c' 的噪声明显小于原密文 c 。

自举能够扩展可执行的计算深度，但计算开销较大。对于乘法深度较小的任务，通常可以直接选择足够的参数完成计算，而不执行自举。本实验中的卷积主要由密文旋转、密文与明文乘法以及密文加法组成，乘法深度仅为 1，因此不需要使用自举。

3.2 BFV 同态加密方案

BFV (Brakerski/Fan-Vercauteren) 是一种面向整数精确计算的格基全同态加密方案，其安全性通常建立在环上带误差学习问题 (Ring Learning With Errors, RLWE) 的困难性基础上。BFV 支持密文加法和密文乘法，并能够通过批处理编码在一个密文中同时存放和处理多个整数，因此适合实现整数矩阵运算和卷积计算。

BFV 的计算主要在多项式环上进行。设

$$R_q = \mathbb{Z}_q[x]/(x^N + 1),$$

其中， N 为多项式环维度， q 为密文模数。对应的明文空间为

$$R_t = \mathbb{Z}_t[x]/(x^N + 1),$$

其中 t 为明文模数。本实验设置

$$t = 65537.$$

BFV 首先通过密钥生成算法产生私钥、公钥以及同态计算所需的评估密钥。私钥通常是一个系数较小的多项式 $s \in R_q$ 。随机选择多项式 $a \in R_q$ 和小噪声多项式 e 后，可以将公钥简化表示为

$$pk = (p_0, p_1) = (-as + e, a) \pmod{q}.$$

为了加密明文 $m \in R_t$ ，首先将其放大到密文模数空间。设明文缩放因子为

$$\Delta = \left\lfloor \frac{q}{t} \right\rfloor.$$

随机选取小多项式 u, e_1, e_2 ，生成密文

$$c = (c_0, c_1),$$

其中

$$c_0 = p_0 u + e_1 + \Delta m \pmod{q},$$

$$c_1 = p_1 u + e_2 \pmod{q}.$$

解密时，使用私钥计算

$$v = c_0 + c_1 s \pmod{q}.$$

代入公钥和密文的表达式，可以得到

$$v = \Delta m + e_{\text{total}} \pmod{q},$$

其中 e_{total} 为加密过程中引入的总噪声。只要噪声没有超过允许范围，就可以通过缩放和舍入恢复明文：

$$m = \left\lfloor \frac{t}{q} v \right\rfloor \pmod{t}.$$

BFV 的同态加法可以直接对密文分量进行相加。设

$$c^{(1)} = (c_0^{(1)}, c_1^{(1)}), \quad c^{(2)} = (c_0^{(2)}, c_1^{(2)}),$$

则同态加法结果为

$$c_{\text{add}} = (c_0^{(1)} + c_0^{(2)}, c_1^{(1)} + c_1^{(2)}) \pmod{q},$$

并满足

$$\text{Dec}(c_{\text{add}}) = m_1 + m_2 \pmod{t}.$$

密文乘法可以得到两个明文的乘积，但会增加密文分量数量和噪声。两个二分量密文相乘后通常得到三分量密文：

$$(d_0, d_1, d_2),$$

解密关系为

$$d_0 + d_1 s + d_2 s^2.$$

为了将密文重新转换为标准的二分量形式，需要利用重线性化密钥执行重线性化操作。连续密文乘法越多，噪声增长越明显，因此 BFV 参数需要根据计算电路的乘法深度进行设置。

本实验使用的是 OpenFHE 中的 BFV-RNS 实现。RNS (Residue Number System, 剩余数系统) 将较大的密文模数 q 分解为若干较小模数的乘积：

$$q = q_0 q_1 \cdots q_L.$$

一个大整数或多项式可以分别在模 $q_0, q_1, \dots, q_L$ 下表示和计算，从而避免直接处理超大整数，提高多项式运算、密钥切换和同态乘法的执行效率。

BFV 还支持批处理编码。通过中国剩余定理，可以将多个整数编码到同一个明文多项式的不同槽位中。设打包明文为

$$\mathbf{m} = (m_0, m_1, \dots, m_{n-1}),$$

则密文与明文权重向量

$$\mathbf{w} = (w_0, w_1, \dots, w_{n-1})$$

进行逐槽乘法后，解密结果满足

$$\text{Dec}(\text{Enc}(\mathbf{m}) \odot \mathbf{w}) = (m_0 w_0, m_1 w_1, \dots, m_{n-1} w_{n-1}).$$

此外，BFV 可以利用旋转评估密钥对打包密文执行槽位旋转，从而改变不同输入元素在槽位中的位置。密文旋转与逐槽乘法、密文加法结合后，可以实现矩阵乘法、卷积和求和等操作。

本实验只对 4×4 输入矩阵进行加密， 3×3 卷积核作为明文权重参与计算。密文卷积主要包含密文旋转、密文-明文乘法和密文加法，不包含连续的密文-密文乘法。因此，实验所需乘法深度较小，不需要执行重线性化和自举。选择 BFV 方案可以保证解密后的整数卷积结果与普通明文卷积结果进行精确比较。

3.3 开源全同态加密库

开源全同态加密库是对 BFV、BGV 和 CKKS 等全同态加密方案的工程化实现。它将复杂的多项式环运算、噪声采样、密钥切换、模数运算和参数管理封装为可直接调用的编程接口，使开发者无需从零实现底层格密码算法，即可完成密钥生成、编码、加密、密文计算和解密等操作。

需要区分全同态加密方案和全同态加密库。BFV、BGV 和 CKKS 属于具体的密码方案，它们规定了明文空间、密文结构以及同态运算方式；OpenFHE、Microsoft SEAL 和 HElib 属于软件库，用于实现一种或多种同态加密方案。其关系可以表示为

同态加密方案 $\rightarrow$ 开源库实现 $\rightarrow$ 密文计算应用.

一个全同态加密库通常提供参数配置、密钥生成、明文编码、批处理打包、加密、解密、同态加法、同态乘法和密文旋转等功能。其基本使用流程为

$$\mathbf{x} \xrightarrow{\text{Encode}} m \xrightarrow{\text{Enc}} c \xrightarrow{\text{Eval}} c_{\text{out}} \xrightarrow{\text{Dec}} m_{\text{out}} \xrightarrow{\text{Decode}} \mathbf{y}.$$

其中，输入数据首先被编码为同态加密方案支持的明文形式，再使用公钥加密为密文。计算方在不知道输入明文和私钥的情况下，对密文执行加法、乘法、旋转等操作，最后由私钥持有者解密并恢复计算结果。正确性要求满足

$$\text{Dec}(sk, \text{Eval}(f, \text{Enc}(pk, m))) = f(m).$$

对于支持批处理的同态加密方案，多个整数可以被打包到同一个密文的不同槽位中。设打包数据为

$$\mathbf{x} = (x_0, x_1, \dots, x_{n-1}),$$

则一次密文运算可以同时作用于多个槽位，具有类似 SIMD 的并行计算特性。密文旋转可以调整数据在槽位中的位置，而逐槽乘法和密文加法可以完成权重计算与结果累加，因此适合实现矩阵运算和卷积计算。

本实验选用 OpenFHE 作为开源全同态加密库，并采用其中的 BFV-RNS 方案。OpenFHE 负责完成密码参数生成、密钥生成、批处理编码、加密、密文旋转、密文-明文乘法、密文加法和解密等操作。实验将 4×4 输入矩阵打包到一个 BFV 密文中，并通过“打包-旋转-累加”策略实现 3×3 卷积核对应的密文卷积。

3.4 “打包 $\rightarrow$ 旋转 $\rightarrow$ 累加”策略

在全同态加密中，如果将输入矩阵的每个元素分别加密，则一次卷积需要处理大量独立密文，计算和存储开销较大。为提高密文槽位利用率，本实验采用“打包-旋转-累加”策略，将整个 4×4 输入矩阵编码到一个 BFV 打包密文中，再通过槽位旋转和逐槽运算实现卷积。

数据打包 首先将输入矩阵按照行优先方式展开：

$$X = \begin{bmatrix} x_0 & x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 & x_7 \\ x_8 & x_9 & x_{10} & x_{11} \\ x_{12} & x_{13} & x_{14} & x_{15} \end{bmatrix},$$

对应的打包向量为

$$\text{Pack}(X) = (x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, x_{12}, x_{13}, x_{14}, x_{15}).$$

完成批处理编码和加密后，得到单个输入密文

$$C_X = \text{Enc}(\text{Pack}(X)).$$

在该布局中，水平方向移动一个元素对应槽位偏移 1，垂直方向移动一行对应槽位偏移 4。因此， 3×3 卷积核中的九个位置对应以下槽位偏移：

$$D = \{0, 1, 2, 4, 5, 6, 8, 9, 10\}.$$

其二维关系可以表示为

$$\begin{bmatrix} 0 & 1 & 2 \\ 4 & 5 & 6 \\ 8 & 9 & 10 \end{bmatrix}.$$

密文旋转与权重乘法 密文旋转用于改变打包数据在槽位中的位置，使卷积窗口中不同位置的输入元素对齐到相同的输出槽位。设 $\text{Rot}(C_X, d)$ 表示将输入密文旋转 d 个槽位，则偏移 d 对应的输入元素可以通过

$$\text{Rot}(C_X, d)$$

获得。

由于旋转是循环置换，可能将不属于合法卷积窗口的数据移动到输出位置，因此每个旋转结果还需要与一个明文权重掩码相乘。设卷积核位置 (a, b) 对应的权重为 $k_{a,b}$ ，其明文掩码为 $M_{a,b}$ ，则

$$M_{a,b} \odot \text{Rot}(C_X, 4a + b)$$

同时完成以下两个操作：

- (1) 将对应输入元素乘以卷积核权重 $k_{a,b}$ ；
- (2) 将无效槽位中的数据清零，仅保留合法输出位置。

本实验将四个卷积输出保存在槽位集合

$$S = \{0, 1, 4, 5\},$$

分别对应

$$y_{00} \rightarrow 0, \quad y_{01} \rightarrow 1, \quad y_{10} \rightarrow 4, \quad y_{11} \rightarrow 5.$$

因此，明文掩码只在槽位 0、1、4 和 5 处设置对应权重，其余槽位均设置为 0。

乘积累加 完成九个卷积核位置对应的旋转和明文乘法后，将所有部分密文相加，即可得到最终卷积结果密文：

$$C_Y = \sum_{a=0}^2 \sum_{b=0}^2 M_{a,b} \odot \text{Rot}(C_X, 4a + b).$$

解密 C_Y 后, 可以在槽位 0、1、4 和 5 中得到四个卷积输出。对于本实验的固定输入, 正确的槽位结果为

$$(348, 393, 0, 0, 528, 573, 0, 0, 0, 0, 0, 0, 0, 0).$$

将有效槽位重新排列后得到

$$Y = \begin{bmatrix} 348 & 393 \\ 528 & 573 \end{bmatrix}.$$

基线方案对偏移 1,2,4,5,6,8,9,10 分别执行旋转。由于偏移 0 可以直接使用原始密文, 因此基线方案共需要 8 次密文旋转、9 次密文-明文乘法和 8 次密文加法。

旋转结果复用 进一步观察九个偏移可以发现

$$\{0, 1, 2, 4, 5, 6, 8, 9, 10\} = \{0, 1, 2\} + \{0, 4, 8\}.$$

其中, $\{0, 1, 2\}$ 表示水平方向偏移, $\{0, 4, 8\}$ 表示垂直方向偏移。因此, 可以先计算

$$R_0 = C_X, \quad R_1 = \text{Rot}(C_X, 1), \quad R_2 = \text{Rot}(C_X, 2),$$

只使用 2 次旋转生成三个水平方向的密文。

随后复用 R_0 、 R_1 和 R_2 , 分别计算卷积核三行对应的部分结果:

$$H_0 = M'_{0,0} \odot R_0 + M'_{0,1} \odot R_1 + M'_{0,2} \odot R_2,$$

$$H_1 = M'_{1,0} \odot R_0 + M'_{1,1} \odot R_1 + M'_{1,2} \odot R_2,$$

$$H_2 = M'_{2,0} \odot R_0 + M'_{2,1} \odot R_1 + M'_{2,2} \odot R_2.$$

最后, 将第二行和第三行中间结果旋转到目标输出位置, 并进行累加:

$$C_Y = H_0 + \text{Rot}(H_1, 4) + \text{Rot}(H_2, 8).$$

该阶段再执行 2 次旋转, 因此优化方案的总旋转次数为

$$2 + 2 = 4.$$

与基线方案相比, 优化方案通过复用水平方向的旋转结果, 避免了对偏移 5、6、9 和 10 分别

执行独立旋转，使旋转次数由 8 次降低至 4 次，减少了

$$\frac{8-4}{8} \times 100\% = 50\%.$$

因此，“打包-旋转-累加”策略的核心是：利用打包编码将多个输入元素放入一个密文，通过旋转完成槽位对齐，通过明文掩码完成权重乘法和边界处理，最后利用密文加法累加各个卷积核位置的贡献；进一步复用具有相同水平偏移的旋转结果，可以显著减少密文旋转次数。

4 明文槽位模拟

4.1 实验输入

本实验实现单输入通道、单输出通道的二维卷积。输入矩阵大小为 4×4 ，卷积核大小为 3×3 ，步长为 1，无填充。实验采用深度学习中常用的卷积计算形式，即卷积核在计算过程中不进行翻转。从严格的数学定义看，该运算属于二维互相关，但在卷积神经网络和相关软件框架中通常仍称为卷积。

固定输入矩阵为

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix},$$

卷积核为

$$K = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}.$$

对于输入矩阵 $X \in \mathbb{Z}^{4 \times 4}$ 和卷积核 $K \in \mathbb{Z}^{3 \times 3}$ ，输出位置 (i, j) 的计算公式为

$$y_{i,j} = \sum_{a=0}^2 \sum_{b=0}^2 x_{i+a,j+b} k_{a,b}, \quad i, j \in \{0, 1\}.$$

由于卷积步长为 1 且不进行填充，输出矩阵的行数和列数均为

$$\frac{4-3}{1} + 1 = 2,$$

因此最终输出为 2×2 矩阵。

以左上角输出元素为例，其计算过程为

$$\begin{aligned}
 y_{00} &= 1 \times 1 + 2 \times 2 + 3 \times 3 \\
 &\quad + 5 \times 4 + 6 \times 5 + 7 \times 6 \\
 &\quad + 9 \times 7 + 10 \times 8 + 11 \times 9 \\
 &= 348.
 \end{aligned} \tag{1}$$

按照相同方法计算其余位置，可得普通明文卷积结果

$$Y = \begin{bmatrix} 348 & 393 \\ 528 & 573 \end{bmatrix}.$$

该结果作为后续密文卷积正确性验证的参考值。

4.2 槽位布局与模拟实现

为了使用 BFV 的批处理编码功能，本实验将输入矩阵按照行优先方式展开为

$$\text{Pack}(X) = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16).$$

各矩阵元素与槽位编号的对应关系为

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{bmatrix}.$$

在该槽位布局下，水平方向相邻元素之间的槽位偏移为 1，垂直方向相邻两行之间的槽位偏移为 4。因此， 3×3 卷积核中的九个元素对应的输入槽位偏移为

$$D = \{0, 1, 2, 4, 5, 6, 8, 9, 10\}.$$

4×4 输入矩阵上共有 4 个合法的 3×3 卷积窗口，其左上角分别位于槽位 0、1、4 和 5。因此，本实验将四个卷积输出保存到槽位集合

$$S = \{0, 1, 4, 5\}.$$

四个输出元素与槽位的对应关系为

$$y_{00} \rightarrow 0, \quad y_{01} \rightarrow 1, \quad y_{10} \rightarrow 4, \quad y_{11} \rightarrow 5.$$

因此，预期的槽位结果为

(348, 393, 0, 0, 528, 573, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0).

将前 16 个槽位重新排列为 4×4 形式，可表示为

$$\begin{bmatrix} 348 & 393 & 0 & 0 \\ 528 & 573 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

其中，槽位 0、1、4 和 5 保存有效卷积结果，其余槽位通过明文掩码设置为 0。

在执行真实的 BFV 密文计算之前，首先使用 Python 程序在普通整数向量上模拟槽位旋转、明文掩码乘法和结果累加。明文槽位模拟的核心代码如下代码 1 所示。

Listing 1: 明文槽位旋转与基线卷积模拟

```
1 OFFSETS = [0, 1, 2, 4, 5, 6, 8, 9, 10]
2 OUTPUT_SLOTS = [0, 1, 4, 5]
3 SLOT_COUNT = 16
4
5 def rotate_left(values, offset):
6     offset %= len(values)
7     return values[offset:] + values[:offset]
8
9 def baseline_convolution_slots(input_slots, kernel):
10    result = [0] * SLOT_COUNT
11
12    for a in range(3):
13        for b in range(3):
14            offset = 4 * a + b
15            shifted = rotate_left(input_slots, offset)
16
17            mask = [0] * SLOT_COUNT
18            for slot in OUTPUT_SLOTS:
19                mask[slot] = kernel[a][b]
20
21            for slot in range(SLOT_COUNT):
22                result[slot] += shifted[slot] * mask[slot]
23
24    return result
```

代码首先按照偏移集合 D 对输入向量进行循环旋转，再构造仅在输出槽位 S 上非零的权重掩码。旋转结果与掩码逐槽相乘后进行累加，其计算语义与后续 OpenFHE 程序中的 `EvalRotate`、`EvalMult` 和 `EvalAdd` 相对应。

基线方案和优化方案的明文槽位模拟结果如图 1 所示。两种方案均得到预期槽位结果，程序最终输出 `reference verification passed`，说明槽位组织、旋转方向、权重设置和累加逻辑均正确。

需要说明的是，该 Python 程序只模拟槽位层面的运算过程，不包括密钥生成、加密、密钥切换和解密。真实的 BFV 密文计算由后续 OpenFHE 程序完成。

```
(OSS CAD Suite) zhongqing@sait12:~/Desktop/fhe_conv_assignment$ python3 scripts/verify_reference.py
naive: [348, 393, 0, 0, 528, 573, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
optimized: [348, 393, 0, 0, 528, 573, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
reference verification passed
```

图 1: 基线方案与优化方案的明文槽位模拟结果

5 作业 5: 基线密文卷积

本实验选择 OpenFHE 作为开源全同态加密库，并采用其中的 BFV-RNS 方案实现整数密文卷积。实验程序使用 C++17 编写，在 CPU 环境中运行。项目文件的组织方式如表 2 所示。

表 2: 实验程序文件组成

文件	作用
include/fhe_convolution.hpp	数据结构和函数接口声明
src/fhe_convolution.cpp	基线与优化密文卷积实现
src/main.cpp	固定测试、随机测试和性能测试
scripts/verify_reference.py	明文槽位逻辑模拟与验证
results/final_run.txt	正式实验运行结果记录

实验的主要参数如表 3 所示。

表 3: BFV-RNS 主要实验参数

参数	设置
全同态加密库	OpenFHE 1.5.1
同态加密方案	BFV-RNS
执行平台	CPU
安全级别	HEStd_128_classic
明文模数	65537
多项式环维度	8192
有效数据槽位数	16
乘法深度	1
密钥切换方式	HYBRID
输入矩阵尺寸	4×4
卷积核尺寸	3×3
卷积步长	1
填充方式	无填充

程序首先创建 BFV-RNS 密码上下文，并启用公钥加密、密钥切换、同态运算和槽位旋转等功能。随后生成公私钥对和旋转评估密钥，将输入矩阵按照行优先方式编码并加密为一个 BFV 打包密文。

完整计算流程可以表示为

$$X \xrightarrow{\text{Pack}} \mathbf{x} \xrightarrow{\text{Encode}} m_X \xrightarrow{\text{Enc}} C_X \xrightarrow{\text{EvalConv}} C_Y \xrightarrow{\text{Dec}} m_Y \xrightarrow{\text{Decode}} Y.$$

BFV 上下文、密钥和输入密文的核心创建过程如代码 2 所示。

Listing 2: BFV-RNS 上下文、密钥和输入密文创建

```
1  CCParams<CryptoContextBFVRNS> parameters;  
2  parameters.SetPlaintextModulus(65537);  
3  parameters.SetBatchSize(16);  
4  parameters.SetMultiplicativeDepth(1);  
5  parameters.SetSecurityLevel(HEStd_128_classic);  
6  parameters.SetKeySwitchTechnique(HYBRID);  
7  
8  CryptoContext<DCRTPoly> context =  
9      GenCryptoContext(parameters);  
10  
11  context->Enable(PKE);  
12  context->Enable(KEYSWITCH);  
13  context->Enable(LEVELEDSSHE);  
14  context->Enable(ADVANCEDSHE);  
15  
16  KeyPair<DCRTPoly> keys = context->KeyGen();  
17  
18  std::vector<int32_t> rotations{  
19      1, 2, 4, 5, 6, 8, 9, 10  
20  };  
21  context->EvalRotateKeyGen(keys.secretKey, rotations);  
22  
23  std::vector<int64_t> packedInput{  
24      1, 2, 3, 4,  
25      5, 6, 7, 8,  
26      9, 10, 11, 12,  
27      13, 14, 15, 16  
28  };  
29  
30  Plaintext inputPlaintext =  
31      context->MakePackedPlaintext(packedInput);  
32  
33  Ciphertext<DCRTPoly> inputCiphertext =  
34      context->Encrypt(keys.publicKey, inputPlaintext);
```

其中，旋转评估密钥用于支持偏移集合

$$\{1, 2, 4, 5, 6, 8, 9, 10\}.$$

虽然优化方案只使用其中的 $\{1, 2, 4, 8\}$ ，但为了在同一个密码上下文中同时运行基线方案和优化方案，程序生成了两种方案所需旋转索引的并集。

程序实际创建的 BFV 参数、固定输入矩阵、卷积核和普通明文卷积结果如图 2 所示。

```

Creating OpenFHE BFV-RNS context and rotation keys...
Ring dimension: 8192
Plaintext modulus: 65537

Input 4x4:
  1   2   3   4
  5   6   7   8
  9  10  11  12
 13  14  15  16

Kernel 3x3:
  1   2   3
  4   5   6
  7   8   9

Plain convolution 2x2:
 348  393
 528  573

Naive decrypted slots:
 348  393  0  0
 528  573  0  0
  0   0  0  0
  0   0  0  0

```

图 2: BFV-RNS 参数、实验输入与明文卷积结果

由图 2 可知，OpenFHE 自动选择的多项式环维度为 8192，明文模数为 65537。固定样例的普通明文卷积结果为

$$Y_{\text{plain}} = \begin{bmatrix} 348 & 393 \\ 528 & 573 \end{bmatrix}.$$

输入矩阵采用行优先方式打包时，水平方向移动一列对应槽位偏移 1，垂直方向移动一行对应槽位偏移 4。因此， 3×3 卷积核中的九个位置对应的槽位偏移集合为

$$D = \{0, 1, 2, 4, 5, 6, 8, 9, 10\}.$$

设输入密文为 C_X ，密文旋转 d 个槽位记为

$$\text{Rot}(C_X, d).$$

卷积核中位置 (a, b) 对应的偏移为

$$d_{a,b} = 4a + b.$$

四个合法卷积输出保存在槽位集合

$$S = \{0, 1, 4, 5\}.$$

对于卷积核位置 (a, b) ，构造明文掩码 $M_{a,b}$ ：

$$M_{a,b}[j] = \begin{cases} k_{a,b}, & j \in S, \\ 0, & j \notin S. \end{cases}$$

将旋转后的密文与明文掩码逐槽相乘，得到

$$P_{a,b} = M_{a,b} \odot \text{Rot}(C_X, 4a + b).$$

该操作一方面将输入槽位乘以对应的卷积核权重，另一方面将无效槽位清零，从而避免循环旋转产生的矩阵行边界回绕和无效槽位污染。

基线密文卷积的核心实现如代码 3 所示。代码中省略了异常处理、函数参数检查和操作次数统计。

Listing 3: 8 次旋转基线密文卷积核心实现

```
1  const std::array<int, 4> outputSlots{0, 1, 4, 5};
2
3  Ciphertext<DCRTPoly> result;
4  bool resultInitialized = false;
5
6  for (int a = 0; a < 3; ++a) {
7      for (int b = 0; b < 3; ++b) {
8          const int offset = 4 * a + b;
9
10         Ciphertext<DCRTPoly> shifted =
11             (offset == 0)
12             ? inputCiphertext
13             : context->EvalRotate(inputCiphertext, offset);
14
15         std::vector<int64_t> mask(16, 0);
16         for (int slot : outputSlots) {
17             mask[slot] = kernel[a][b];
18         }
19
20         Plaintext maskPlaintext =
21             context->MakePackedPlaintext(mask);
22
23         Ciphertext<DCRTPoly> term =
24             context->EvalMult(shifted, maskPlaintext);
25
26         if (!resultInitialized) {
27             result = term;
28             resultInitialized = true;
29         } else {
30             result = context->EvalAdd(result, term);
31         }
32     }
33 }
```

将九个卷积位置对应的部分结果累加，得到

$$C_Y^{\text{base}} = \sum_{a=0}^2 \sum_{b=0}^2 M_{a,b} \odot \text{Rot}(C_X, 4a + b).$$

偏移 0 直接使用原始密文，不需要实际旋转；其余非零偏移为

$$\{1, 2, 4, 5, 6, 8, 9, 10\}.$$

因此基线方案需要 8 次密文旋转。九个卷积核权重对应 9 次密文-明文乘法，九个部分结果累加需要 8 次密文加法，即

$$N_{\text{rot}}^{\text{base}} = 8, \quad N_{\text{mul}}^{\text{base}} = 9, \quad N_{\text{add}}^{\text{base}} = 8.$$

该实现只使用密文-明文乘法，没有连续的密文-密文乘法，因此乘法深度较小，不需要执行自举。

6 作业 6：旋转次数优化与理论分析

在全同态加密中，密文旋转通常需要执行密钥切换，计算开销较大。因此，在密文-明文乘法次数和密文加法次数不变的情况下，减少旋转次数能够降低密文卷积的整体执行时间。

基线方案对八个非零偏移分别执行旋转，没有复用不同偏移之间的结构关系。观察偏移集合可得

$$D = \{0, 1, 2, 4, 5, 6, 8, 9, 10\} = \{0, 1, 2\} + \{0, 4, 8\}.$$

其中， $\{0, 1, 2\}$ 表示卷积核列方向的水平偏移， $\{0, 4, 8\}$ 表示卷积核行方向的垂直偏移。因此，可以先生成水平方向的三个中间密文：

$$R_0 = C_X, \quad R_1 = \text{Rot}(C_X, 1), \quad R_2 = \text{Rot}(C_X, 2).$$

其中 R_0 直接使用原始密文，因此水平阶段只需要 2 次旋转。

对于卷积核第 a 行，定义明文掩码 $M'_{a,b}$ ，其非零位置为

$$S + 4a = \{4a, 4a + 1, 4a + 4, 4a + 5\}.$$

利用已经生成的 R_0 、 R_1 和 R_2 构造第 a 行的部分卷积结果：

$$H_a = \sum_{b=0}^2 M'_{a,b} \odot R_b.$$

三个行部分结果分别为

$$H_0 = M'_{0,0} \odot R_0 + M'_{0,1} \odot R_1 + M'_{0,2} \odot R_2,$$

$$H_1 = M'_{1,0} \odot R_0 + M'_{1,1} \odot R_1 + M'_{1,2} \odot R_2,$$

$$H_2 = M'_{2,0} \odot R_0 + M'_{2,1} \odot R_1 + M'_{2,2} \odot R_2.$$

此时, H_0 已经位于最终输出槽位, H_1 和 H_2 分别保存在向下偏移 4 和 8 个槽位的位置。最终执行

$$C_Y^{\text{opt}} = H_0 + \text{Rot}(H_1, 4) + \text{Rot}(H_2, 8).$$

优化方案的核心代码如代码 4 所示。

Listing 4: 4 次旋转优化密文卷积核心实现

```
1  const std::array<int, 4> outputSlots{0, 1, 4, 5};
2
3  std::array<Ciphertext<DCRTPoly>, 3> horizontal;
4  horizontal[0] = inputCiphertext;
5  horizontal[1] = context->EvalRotate(inputCiphertext, 1);
6  horizontal[2] = context->EvalRotate(inputCiphertext, 2);
7
8  std::array<Ciphertext<DCRTPoly>, 3> rowResults;
9
10 for (int a = 0; a < 3; ++a) {
11     bool rowInitialized = false;
12
13     for (int b = 0; b < 3; ++b) {
14         std::vector<int64_t> mask(16, 0);
15
16         for (int slot : outputSlots) {
17             mask[slot + 4 * a] = kernel[a][b];
18         }
19
20         Plaintext maskPlaintext =
21             context->MakePackedPlaintext(mask);
22
23         Ciphertext<DCRTPoly> term =
24             context->EvalMult(horizontal[b], maskPlaintext);
25
26         if (!rowInitialized) {
27             rowResults[a] = term;
28             rowInitialized = true;
29         } else {
30             rowResults[a] =
31                 context->EvalAdd(rowResults[a], term);
32         }
33     }
34 }
35
```

```

36 Ciphertext<DCRTPoly> alignedRow1 =
37     context->EvalRotate(rowResults[1], 4);
38
39 Ciphertext<DCRTPoly> alignedRow2 =
40     context->EvalRotate(rowResults[2], 8);
41
42 Ciphertext<DCRTPoly> result =
43     context->EvalAdd(rowResults[0], alignedRow1);
44
45 result = context->EvalAdd(result, alignedRow2);

```

该代码首先执行偏移 1 和 2 两次旋转，生成三个可复用的水平密文。随后分别完成卷积核三行的密文-明文乘法与行内累加，最后再执行偏移 4 和 8 两次旋转，将第二、三行结果对齐到输出槽位。

因此，优化方案的旋转次数为

$$N_{\text{rot}}^{\text{opt}} = 2 + 2 = 4.$$

优化版本实际使用的旋转索引为

$$\{1, 2, 4, 8\},$$

基线版本使用的旋转索引为

$$\{1, 2, 4, 5, 6, 8, 9, 10\}.$$

优化方案避免了对偏移 5、6、9 和 10 分别生成独立旋转密文。两种方案仍然需要 9 次密文-明文乘法和 8 次密文加法，因此

$$N_{\text{rot}}^{\text{opt}} = 4, \quad N_{\text{mul}}^{\text{opt}} = 9, \quad N_{\text{add}}^{\text{opt}} = 8.$$

旋转次数由 8 次降低为 4 次，下降比例为

$$\frac{8 - 4}{8} \times 100\% = 50\%.$$

为了分析 4 次旋转是否达到理论最小值，考虑本文采用的计算模型：

1. 输入由一个打包密文表示；
2. 允许使用明文对角掩码乘法；
3. 允许使用密文加法和密文旋转；
4. 卷积核为一般稠密 3×3 卷积核。

一个输出槽位需要依赖九个不同的输入偏移：

$$\{0, 1, 2, 4, 5, 6, 8, 9, 10\}.$$

明文掩码乘法只能改变已有输入来源的权重，不能产生新的输入偏移；密文加法只能合并已有的输入来源。在本文采用的可复用旋转分支模型下，一次旋转最多使可组合的输入依赖路径数量翻倍。因此，执行 r 次旋转后，一个输出槽位最多形成

$$2^r$$

个不同的输入依赖路径。

若只使用 3 次旋转，则最多得到

$$2^3 = 8$$

个输入依赖，无法覆盖稠密 3×3 卷积所需的 9 个位置。因此必须满足

$$2^r \geq 9,$$

即

$$r \geq \lceil \log_2 9 \rceil = 4.$$

本实验优化方案恰好使用 4 次旋转，因此在本文限定的单输入打包密文、明文对角乘法、密文加法和可复用密文旋转模型下，达到了稠密 3×3 卷积的理论最小旋转次数。

该结论具有明确的适用范围。若卷积核为稀疏卷积核、改变输入打包布局、增加输入密文数量，或者引入其他特殊预处理方式，最小旋转次数可能发生变化。

7 实验结果与分析

7.1 正确性验证

程序首先计算普通明文卷积，然后分别执行 8 次旋转的基线算法和 4 次旋转的优化算法。密文计算完成后，使用私钥解密结果并读取前 16 个槽位。

固定样例的解密结果如图 3 所示。

```
Optimized decrypted slots:
  348    393    0    0
  528    573    0    0
    0     0     0     0
    0     0     0     0

Operation counts
method,rotations,ct-pt multiplications,additions
naive,8,9,8
optimized,4,9,8
Random correctness tests: 20/20 passed

Average encrypted convolution time over 30 rounds
naive_ms=39.686
optimized_ms=26.767
speedup=1.483x

All correctness checks passed.
```

图 3: 基线版本与优化版本的密文卷积解密结果

基线方案和优化方案的解密槽位均为

$$(348, 393, 0, 0, 528, 573, 0, 0, 0, 0, 0, 0, 0, 0, 0).$$

其中，有效输出保存在槽位

$$S = \{0, 1, 4, 5\}.$$

提取并重新排列这些槽位后，两种密文卷积均得到

$$Y_{\text{cipher}} = \begin{bmatrix} 348 & 393 \\ 528 & 573 \end{bmatrix}.$$

该结果与普通明文卷积结果完全一致，因此有

$$Y_{\text{plain}} = Y_{\text{base}} = Y_{\text{opt}}.$$

为了避免固定样例带来的偶然性，程序还使用固定随机种子生成 20 组输入矩阵和卷积核。随机样例中包含正数、负数和零，并分别比较普通明文卷积、基线密文卷积和优化密文卷积的结果。

对于每组测试，程序均验证

$$Y_{\text{plain}}^{(i)} = Y_{\text{base}}^{(i)} = Y_{\text{opt}}^{(i)}, \quad i = 1, 2, \dots, 20.$$

最终输出

Random correctness tests: 20/20 passed,

说明 20 组随机样例全部通过，进一步证明两种密文卷积实现均具有正确性。

7.2 操作次数与运行时间

两种方案的操作次数如表 4 所示。

表 4: 基线方案与优化方案的同态操作次数

方法	密文旋转次数	密文-明文乘法次数	密文加法次数
基线实现	8	9	8
优化实现	4	9	8

优化前后密文-明文乘法次数和密文加法次数保持不变，密文旋转次数由 8 次降低为 4 次，下降 50%。优化过程只改变了中间密文的构造和复用方式，没有改变最终计算的卷积函数。

程序在一次预热后，分别重复执行两种密文卷积 30 次，并计算平均运行时间。操作次数、随机测试和性能结果如图 4 所示。

```
Operation counts
method,rotations,ct-pt multiplications,additions
naive,8,9,8
optimized,4,9,8
Random correctness tests: 20/20 passed

Average encrypted convolution time over 30 rounds
naive_ms=39.686
optimized_ms=26.767
speedup=1.483x
```

图 4: 操作次数、随机正确性测试与运行时间结果

本机运行结果如表 5 所示。

表 5: 密文卷积运行时间比较

方法	平均运行时间/ms	相对加速比
基线实现	39.686	1.000
优化实现	26.767	1.483

优化后的运行时间下降比例为

$$\frac{39.686 - 26.767}{39.686} \times 100\% \approx 32.55\%.$$

基线方案平均运行时间为 39.686 ms，优化方案平均运行时间为 26.767 ms，优化方案获得约 1.483 倍加速。

旋转次数减少了 50%，而总运行时间下降约 32.55%，两者没有完全保持同比例。这是因为程序总运行时间不仅包含密文旋转，还包含 9 次密文-明文乘法、8 次密文加法、中间密文对象创建、内存管理以及 OpenFHE 内部多项式运算等开销。

程序最终输出

All correctness checks passed,

说明固定样例、随机样例、基线实现和优化实现的全部正确性检查均通过。

8 结论

本实验使用 OpenFHE 开源全同态加密库中的 BFV-RNS 方案，实现了单输入通道、单输出通道的 4×4 输入与 3×3 卷积核的精确密文卷积。卷积步长为 1，不进行填充，最终输出为 2×2 矩阵。

作业 5 采用基线密文卷积方法，将输入矩阵按照行优先方式打包到一个 BFV 密文中，并根据九个卷积位置对应的槽位偏移

$$\{0, 1, 2, 4, 5, 6, 8, 9, 10\}$$

分别构造旋转结果。偏移 0 直接使用原始密文，其余八个偏移分别执行一次旋转，因此基线方案共使用 8 次密文旋转、9 次密文-明文乘法和 8 次密文加法。

作业 6 利用偏移集合的二维分解结构

$$\{0, 1, 2, 4, 5, 6, 8, 9, 10\} = \{0, 1, 2\} + \{0, 4, 8\},$$

先通过偏移 1 和 2 生成水平方向的中间密文，再复用这些结果构造卷积核三行的部分和，最后通过偏移 4 和 8 完成垂直方向对齐。该方法将旋转次数由 8 次降低为 4 次，减少 50%，而乘法次数和加法次数保持不变。

理论分析表明，在单输入打包密文、明文对角乘法、密文加法和密文旋转组成的线性计算模型下，稠密 3×3 卷积需要依赖 9 个不同输入位置，而 3 次旋转最多形成 8 个不同输入来源，因此至少需要 4 次旋转。本实验的优化方案恰好使用 4 次旋转，在上述计算模型和一般稠密卷积核条件下达到理论最小值。

固定样例中，普通明文卷积、基线密文卷积和优化密文卷积均得到

$$\begin{bmatrix} 348 & 393 \\ 528 & 573 \end{bmatrix}.$$

20 组随机测试全部通过。在 30 轮性能测试中，基线方案平均运行时间为 39.686 ms，优化方案平均运行时间为 26.767 ms，取得 1.483 倍加速。实验结果说明，合理设计槽位布局并复用中间旋转结果，可以在保证密文卷积正确性的同时，有效减少旋转操作并降低整体计算开销。